Título:	**Programação Orientada a Objetos**
Professor:	Eleandro Maschio
Ano:	2023
Disciplina:	Fundamentos de Programação
Curso:	Sistemas para Internet
Câmpus:	Guarapuava
Instituição:	Universidade Tecnológica Federal do Paraná
Como citar:	Referência





PROGRAMAÇÃO

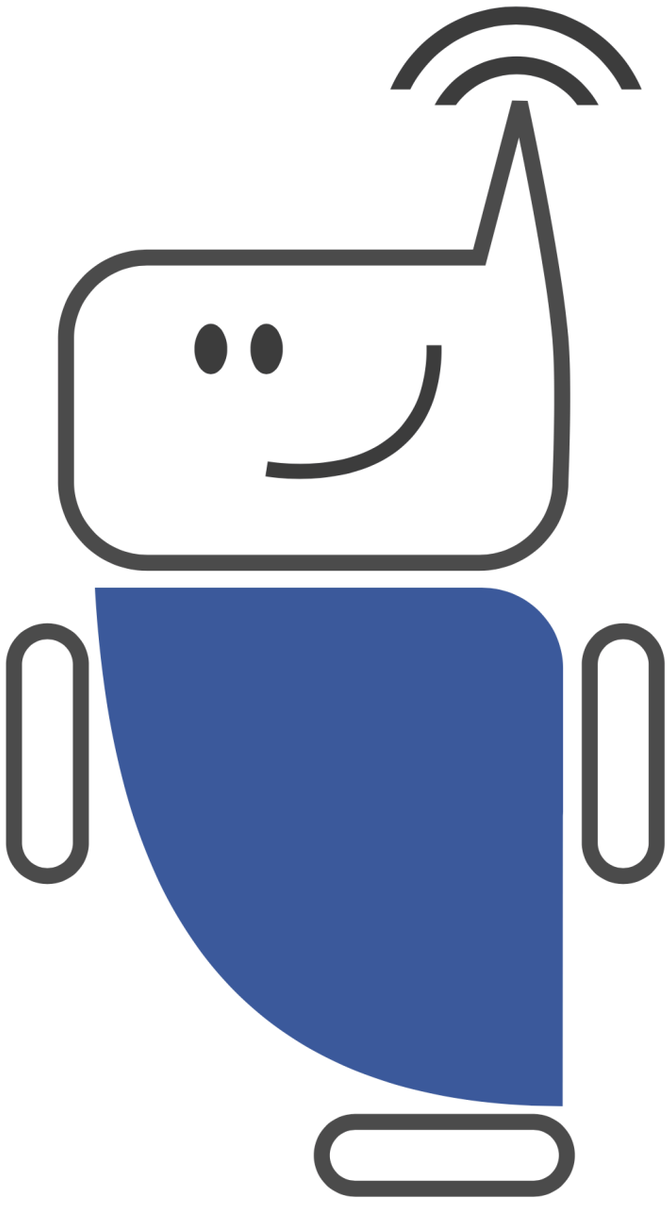
ORIENTADA A OBJETOS



PROF. ELEANDRO MASCHIO

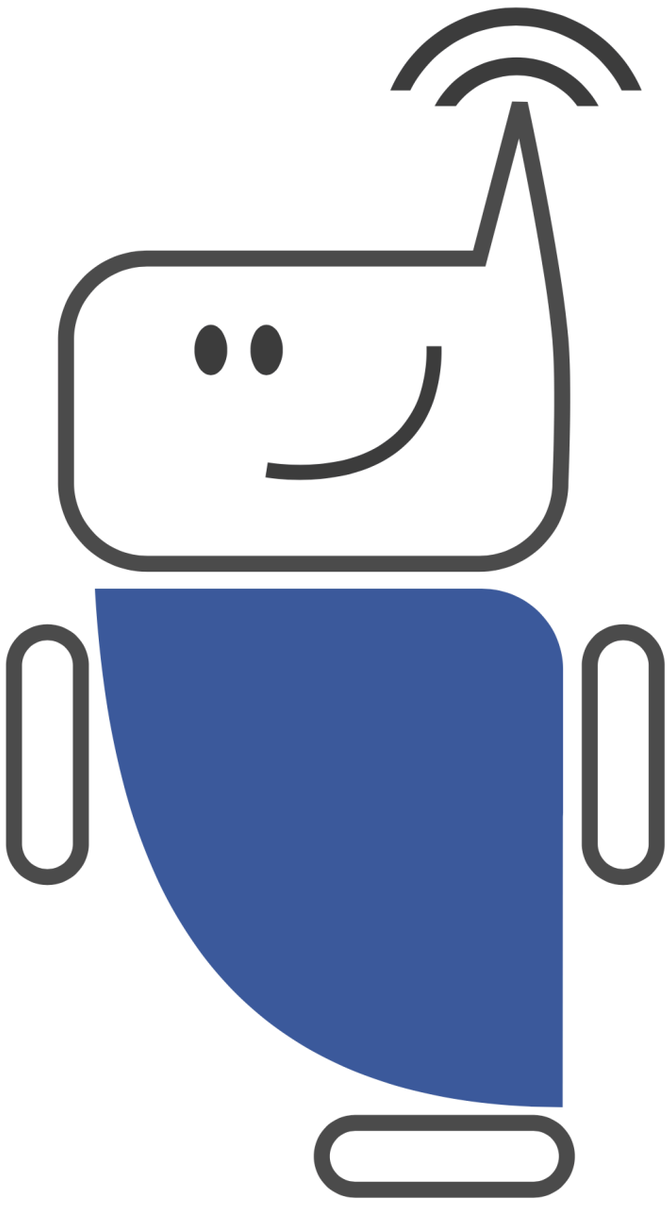
ESTA NÃO É UMA AULA DA DISCIPLINA DE POO!

O OBJETIVO É ENFATIZAR OS ASPECTOS DA ORIENTAÇÃO A OBJETOS MAIS IMPORTANTES PARA FUNDAMENTOS DE PROGRAMAÇÃO.



ROTEIRO

- Além do **`index.ts`**
- Programação Imperativa e Orientada a Objetos
- Terminologia e Conceitos
- Interação com Objetos
- Testes
- Objetos em Representação Textual
- Primeira Modelagens em um Roteiro
- Próximos Passos

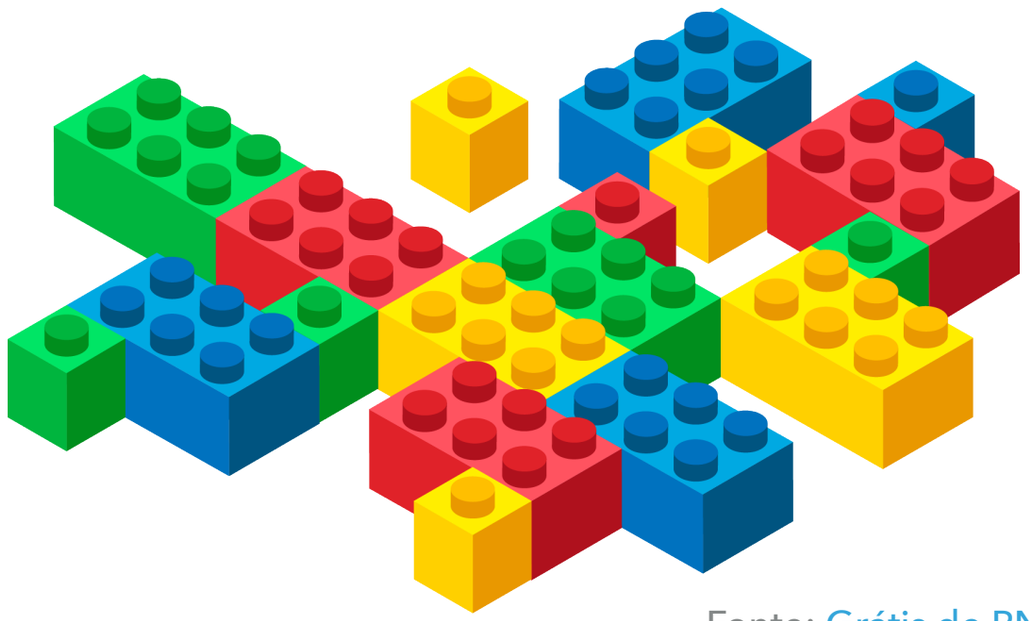


ALÉM DO

INDEX.TS

ATÉ AGORA...

- O principal enfoque da disciplina foi o entendimento das **instruções isoladas**.
- Era necessário aprender como funcionavam **elementos básicos** da programação, como:
  - Estrutura de um código-fonte em TypeScript.
  - Variáveis e constantes.
  - Tipos de dados.
  - Instruções de entrada e de saída.
  - Atribuição.
  - Operações aritméticas.
  - Comentários.



Fonte: Grátis de PNG

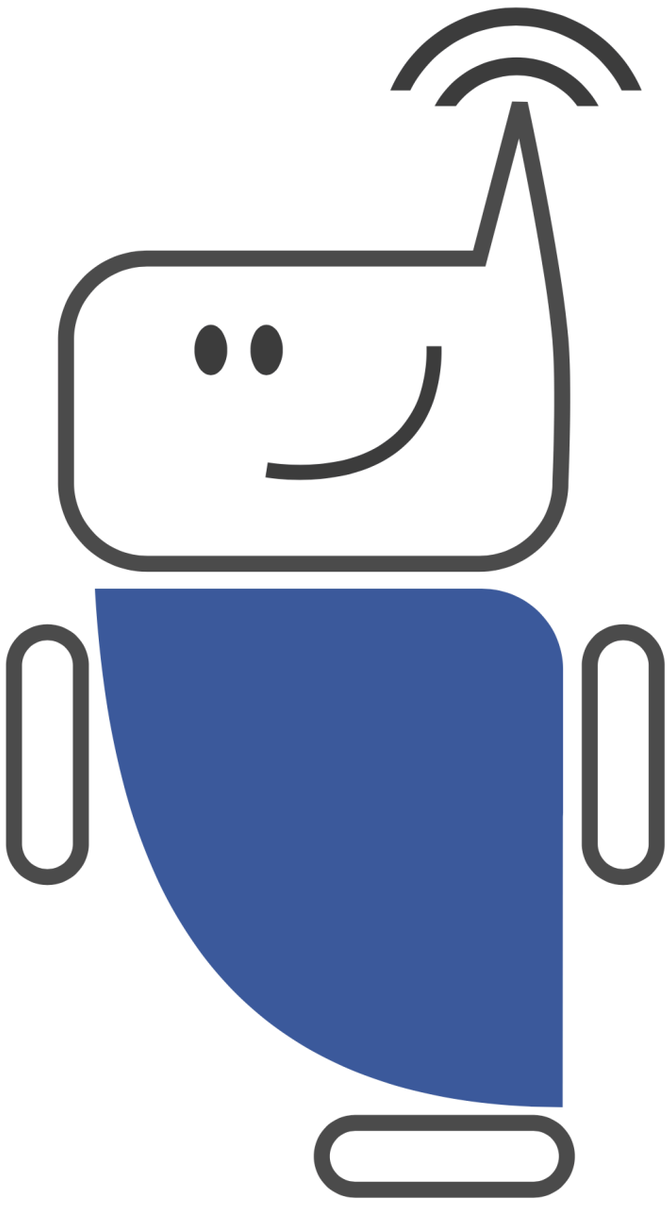
- Favoreceu-se uma **exploração** que proporcionasse o **domínio** desses elementos.

ATÉ AGORA...

- Todos os códigos foram estruturados sequencialmente dentro do arquivo `index.ts`.
  - Funcionou como uma caixa de areia (*sandbox*) para a **exploração das instruções**.
- Mas, nem de longe, esses códigos seguem os princípios do **Paradigma Orientado a Objetos**.
- Isso reforça o que foi destacado na aula de **Introdução aos Aplicativos TypeScript**.
  - Embora TypeScript seja uma linguagem orientada a objetos, simplesmente **codificar em TypeScript não é sinônimo de programar orientado a objetos**.

UMA DIRETRIZ PARA AGORA EM DIANTE...

≈ A PROGRAMAÇÃO ORIENTADA A OBJETOS ESTÁ ALÉM DO INDEX.TS!



PROGRAMAÇÃO

IMPERATIVA E ORIENTADA A OBJETOS

EXEMPLO – DOSADOR

Modele e implemente um dosador automático para um medicamento destinado a idosos. A dosagem é prescrita conforme o peso do paciente (informado ao dosador), sendo de 1 ml para cada 10 kg. O dosador funciona com mililitros e não faz fracionamento. Portanto, uma pessoa de 55 kg recebe uma dosagem de 6 ml.

IMPLEMENTAÇÃO EM INDEX.TS

```
import teclado from "npm:readline-sync";
let peso: number = 0,
  dosagem: number = 0;

console.log("Digite o peso em kg:");
peso = teclado.questionFloat();

dosagem = Math.ceil(peso / 10);
// arredonda para cima e retorna como double.

console.log("Dosagem:", dosagem, "ml");
```

IMPLEMENTAÇÃO NA LINGUAGEM C

```c
#include <stdio.h>
#include <math.h>

int main()
{
   float peso = 0.0f;
   int dosagem = 0;

   printf("Digite o peso em kg: ");
   scanf("%f", &peso);

   dosagem = (int) ceil(peso / 10);
   // arredonda para cima e retorna como double.

   printf("Dosagem: %d ml\n", dosagem);

   return(0);
}
```

Online C Compiler

IMPLEMENTAÇÃO IMPERATIVA EM JAVA

```java
import java.util.Scanner;

public class Dosador
{
   public static void main (String[] args)
   {
      Scanner entrada = new Scanner(System.in);

      float peso = 0.0f;
      int dosagem = 0;

      System.out.print("Digite o peso em kg: ");
      peso = entrada.nextFloat();

      dosagem = (int) Math.ceil(peso / 10);
      // arredonda para cima e retorna como double.

      System.out.println("Dosagem: " + dosagem + " ml");
   }
}
```

Online Java Compiler

IMPLEMENTAÇÃO ESSENCIALMENTE IMPERATIVA

- Essa semelhança, que parece uma **tradução entre linguagens**, evidencia um problema das implementações:
  - Em C, usa **Programação Imperativa**.
  - Em Java, feita inteira dentro do método `main()`, acaba sendo essencialmente imperativa.
  - Em TypeScript, feita inteira e sequencialmente dentro do `index.ts`, também não é orientada a objetos.

- Todas as implementações são focadas no **sequenciamento das instruções**.

IMPLEMENTAÇÃO ESSENCIALMENTE IMPERATIVA

- **Programação Imperativa**
  - Tem a sequência de instruções como foco da modelagem.
- **Programação Orientada a Objetos**
  - Tem o dado como foco da modelagem (o que não estava acontecendo).

- Deve-se modelar o problema em termos de objetos...
  - Com características (atributos).
  - Com comportamentos (métodos).
  - E que se relacionam com outros objetos.

IMPLEMENTAÇÃO ORIENTADA A OBJETOS

```
export default class Dosador {
  private peso: number;

  public constructor(peso: number) {
    this.peso = peso;
  }

  public getPeso(): number {
    return this.peso;
  }

  public setPeso(peso: number): void {
    this.peso = peso;
  }

  public getDosagem(): number {
    return (Math.ceil(this.peso / 10));
  }
}
```

IMPLEMENTAÇÃO ORIENTADA A OBJETOS

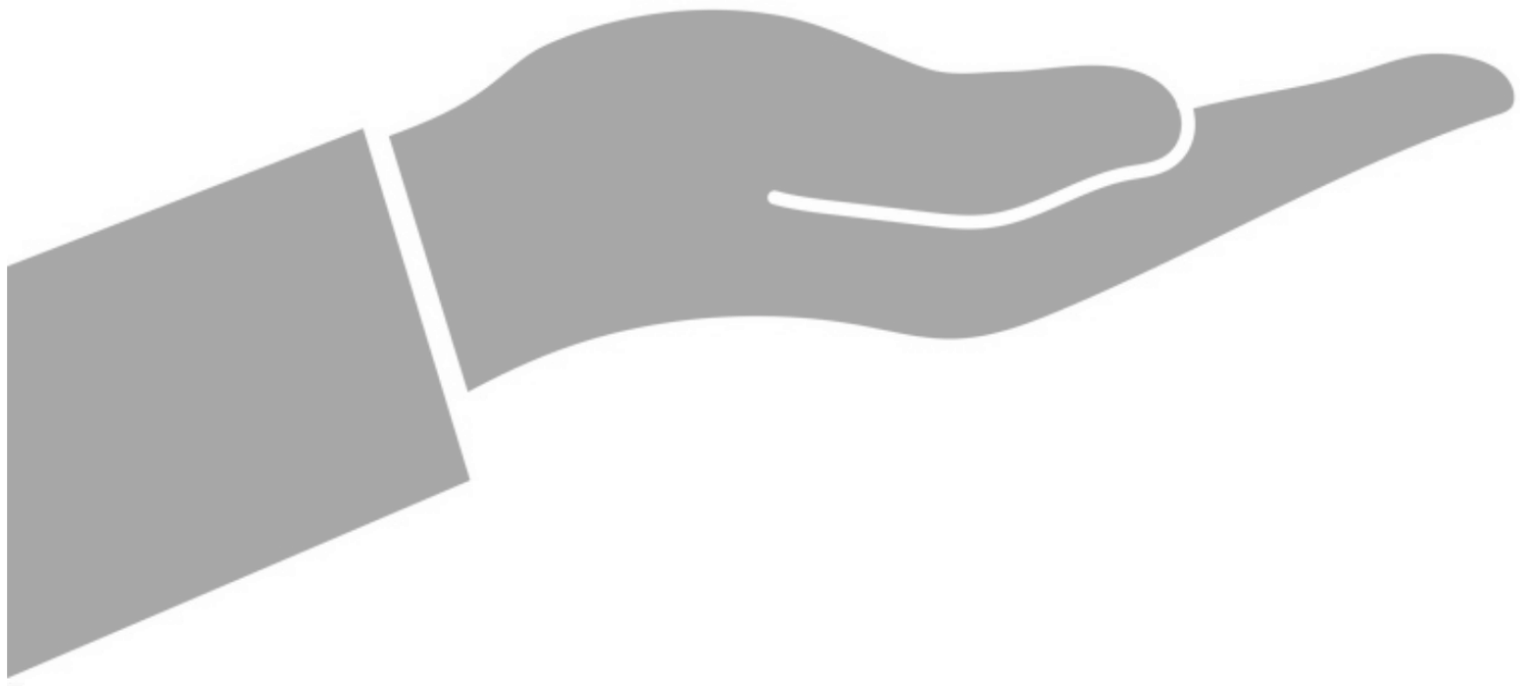
OBJETOS DEVEM SER IMAGINADOS COMO ALGO TANGÍVEL

IMPLEMENTAÇÃO ORIENTADA A OBJETOS

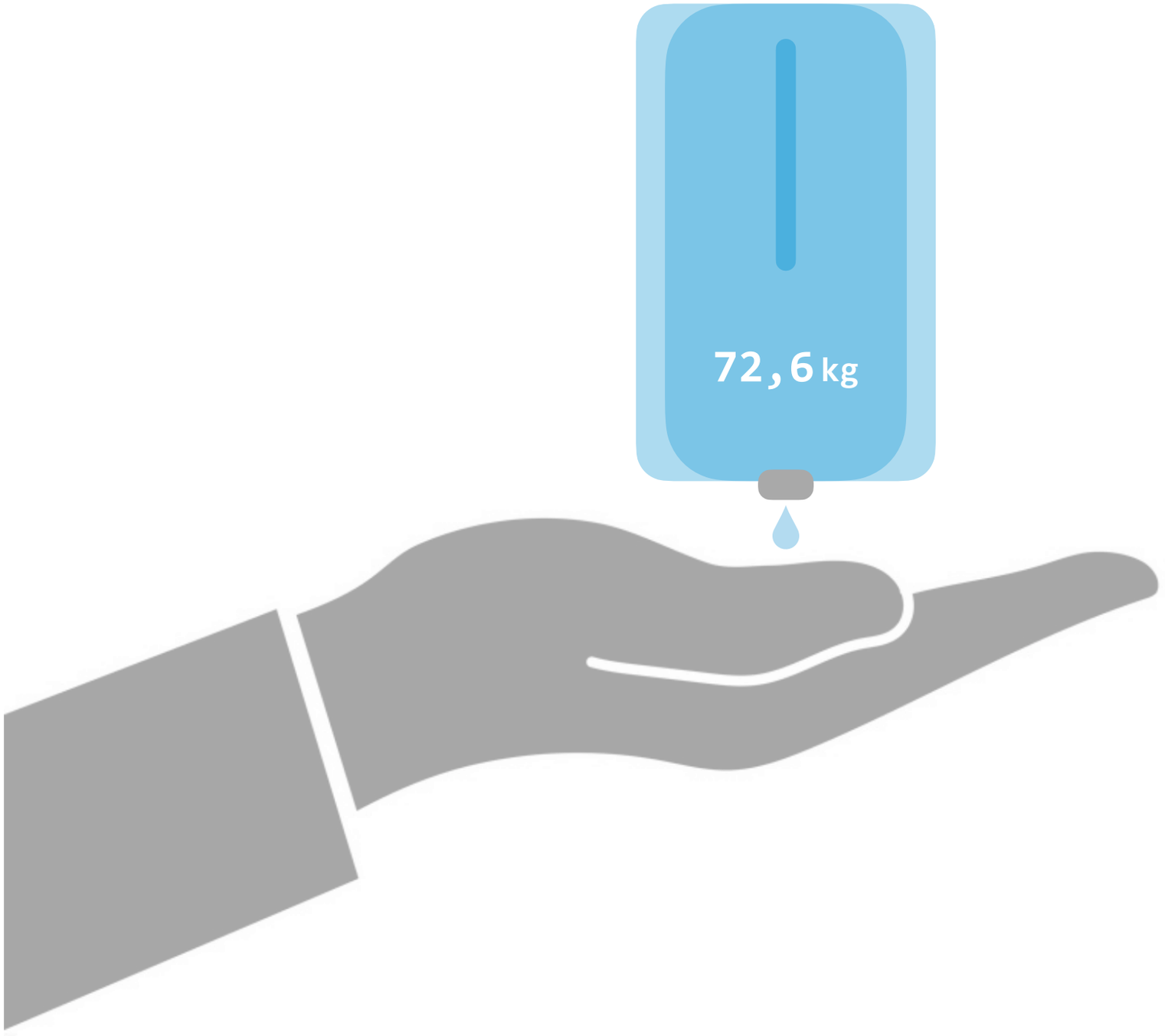
DOSADOR

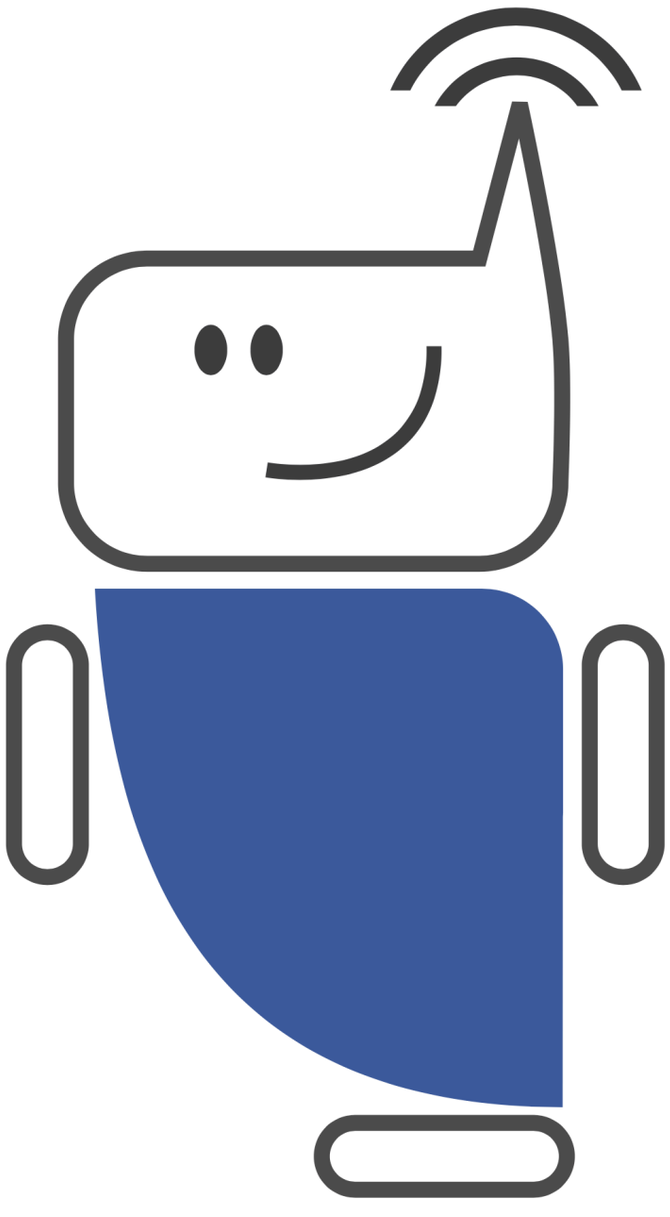
```
- peso: number = 0.0
```

```
+ constructor(peso: number)
+ getPeso(): number
+ setPeso(peso: number): void
+ getDosagem(): number
```



IMPLEMENTAÇÃO ORIENTADA A OBJETOS





TERMINOLOGIA E

CONCEITOS

CLASSE

- É uma descrição que **abstrai um conjunto de objetos** com **características similares**.
- Encapsula **abstrações de dados e de procedimentos** que descrevem características e comportamentos de entidades do mundo real.
  - Essas abstrações do mundo real são representadas por objetos.
- Uma classe descreve:
  - **Características ou estados possíveis** de um conjunto de objetos; e
  - **Comportamentos ou ações aplicáveis** a esses objetos.
- Objetos são criados por meio de classes.

OBJETO

- Representação computacional de qualquer **coisa física ou abstrata**.
  - Pessoa, carro, computador, caixa automático.
  - Conta corrente, matrícula, empréstimo, histórico.
- Objetos possuem, além de atributos e comportamentos:
  - **Identidade**
    - Cada objeto é **único**, considerando a própria identidade.
    - Mudanças de estado de um objeto, exceto se especificado, **não afetam outro**.
  - **Estado**
    - Representa a **situação atual** dos atributos de um objeto em determinado momento.
    - O estado de um objeto pode **mudar ao longo do tempo**.

ATRIBUTOS

- São as **características** da entidade que um objeto representa.
- Exprimem as **particularidades** de um objeto.

- Exemplos:
  - Pessoa: nome, data de nascimento, idade, altura, peso.
  - Carro: modelo, ano de fabricação, cor.
  - Conta corrente: agência, número, dígito verificador, titular.
  - Matrícula: aluno, ano, semestre, disciplina.

MÉTODOS

- São os **comportamentos** da entidade que um objeto representa.
- Exprimem as **ações** que um objeto pode desempenhar ou proporcionar.
- Modelam a **interação** (relacionamento) com outros objetos.

- Exemplos:
  - Pessoa: fazer aniversário, crescer, mudar o peso, cumprimentar outra pessoa.
  - Carro: acelerar, frear, trocar de marcha, abastecer.
  - Conta corrente: sacar, depositar, fornecer empréstimo.
  - Matrícula: efetivar, cancelar, alterar.

INSTÂNCIA

- Um objeto é uma **instância de uma classe**.
- Embora um objeto possa ser visto de maneira genérica, uma instância remete a um **objeto específico com as devidas particularidades**.
- A **identidade** do objeto se manifesta.

- Exemplos:
  - Pessoa: Bart Simpson, 01/04/1979, 10 anos de idade, 1,22m de altura, 38,5kg.
  - Carro: Porsche 911 Carrera, 2024, preto.
  - Conta corrente: agência: 123, número: 4567, dígito verificador: 8.
  - Matrícula: Sheldon Cooper, 2025, 1° semestre, Fundamentos de Programação.

ENCAPSULAMENTO

- Uma classe encapsula **atributos e métodos** que descrevem características e comportamentos de entidades do mundo real.
- Atributos são protegidos de alteração externa por meio de **modificadores de acesso**.
- A interação entre objetos deve ocorrer **exclusivamente** por meio de métodos.
- Caso se queira **alterar um atributo ou invocar uma operação** de um objeto...
  - Deve-se fazer isso utilizando métodos como interface.
  - Funcionam como um protocolo para regrar essas operações.
- Exemplo:
  - Ao invés de mudar, externamente a idade de uma pessoa, invoca-se o método `fazAniversario()`.

CONSTRUTOR

- Responsável pela **inicialização** dos objetos.
  - Valores iniciais pré-estabelecidos; ou
  - Passados por parâmetro.
- É invocado automaticamente quando se instancia um objeto (palavra-reservada `new`).
- Parece um método... porém, a literatura mais ortodoxa não considera o construtor como um método.
- Exemplo:
  - Pessoa: ser inicializada com o nome, peso e altura informados e considerar a data de nascimento como o dia atual.

METÁFORA

- **Classe**
  - Uma forma de bolo.
- **Objeto**
  - Um bolo que saiu da forma.
- **Instância**
  - Bolo de chocolate, bolo de fubá, bolo floresta negra.

METÁFORA

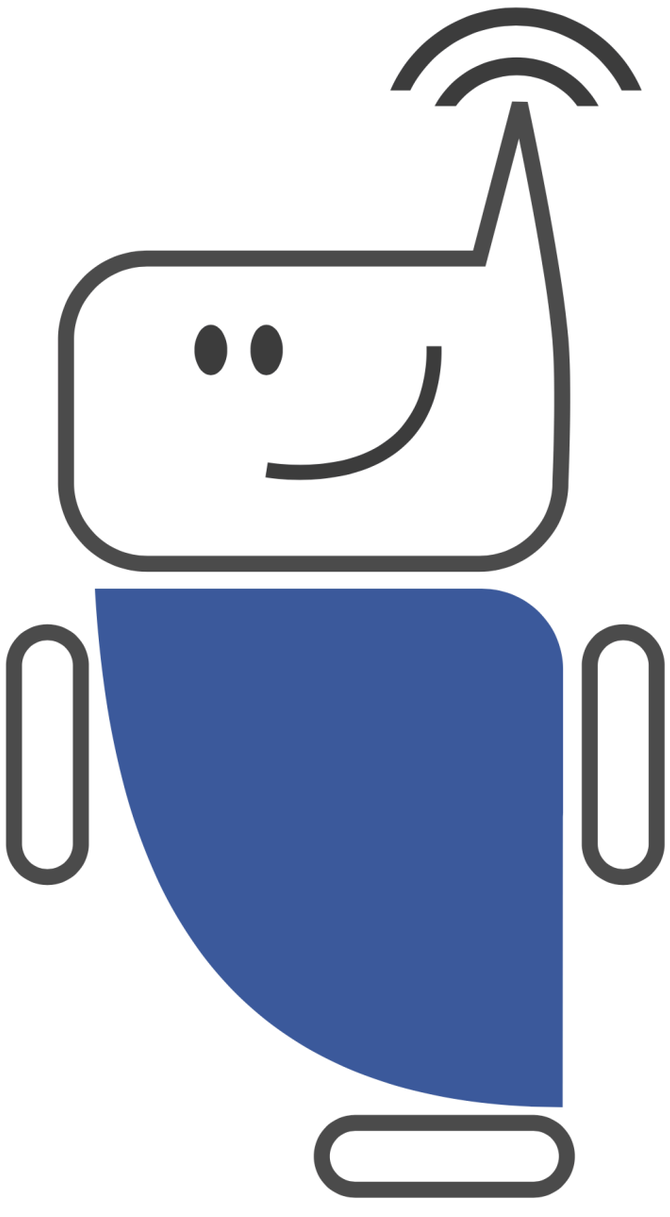
- **Atributos**
  - Peso, formato, tipo da massa, recheio, cobertura.

- **Métodos**
  - Cortar fatia, dividir por um número de pessoas, acrescentar enfeite.

METÁFORA

- **Encapsulamento**
  - Posso mudar o tipo da massa de um bolo pronto?

- **Construtor**
  - Processo de se fazer o bolo.



INTERAÇÃO COM OBJETOS

SOLUÇÕES INTEIRAS IMPLEMENTADAS EM INDEX.TS

- A interação ocorria por meio do teclado e da tela.
  - Instruções de entrada.
  - Instruções de saída.

— VS. —

CLASSES QUE MODELAM A LÓGICA DA SOLUÇÃO

- A interação ocorre por meio dos métodos.
  - Passagem de parâmetros.
  - Retorno.

SOLUÇÕES INTEIRAS IMPLEMENTADAS EM INDEX.TS

A interação ocorria por meio do teclado e da tela.

- **Instruções de entrada** (atribuições com os métodos)
  - `questionInt()`
  - `questionFloat()`
  - `question().charAt(0)`
  - `question()`
- **Instruções de saída**
  - `console.log()`

CLASSES QUE MODELAM A LÓGICA DA SOLUÇÃO

A interação com objetos ocorre por meio dos métodos.

- **Passagem de parâmetros**
  - Traz informações externas ao objeto.
  - Podem ser passadas ao construtor e valorarem atributos.
- **Retorno**
  - Leva informações do objeto ao meio externo.

- Os **parâmetros** e os **retornos** dos métodos são grandes responsáveis por uma **interação (entre objetos)** fluída em uma modelagem.
  - Desenvolver essa habilidade contribui muito para o sucesso em POO.

RETORNOS DE MÉTODOS

- Ao invés de saídas em tela, **classes que modelam a lógica da solução** devem retornar valores por meio de **métodos**.

- Por exemplo, um cálculo esperado que resulte em um inteiro:
  - Deve estar em um **método que retorne** um `number`.
  - Ao invés de ter impresso o número inteiro na tela.
  - Sem falar que esse `number` retornado pode ser **usado ao longo do código**.
  - Um número impresso, em contrapartida, não pode ser capturado novamente da tela.

- Deve-se pensar na **interação entre objetos**.
  - Troca de mensagens e encapsulamento.

RETORNOS DE MÉTODOS E STRINGS FORMATADAS

- Embora seja fácil que os métodos retornem *strings* formatadas, essa **não é a melhor opção**, na maioria das vezes.
- Acaba sendo mais útil que os métodos retornem **apenas os valores** em si.
  - `number`, `boolean`.
  - Ou mesmo `string`, mas sem formatação.
- O motivo é que uma `string` formatada, em geral, somente atende a uma **necessidade específica**.
- Em contrapartida, os valores primitivos podem ser **amplamente usados** ao longo do código.
  - Cálculos, expressões condicionais e outras saídas diversas.

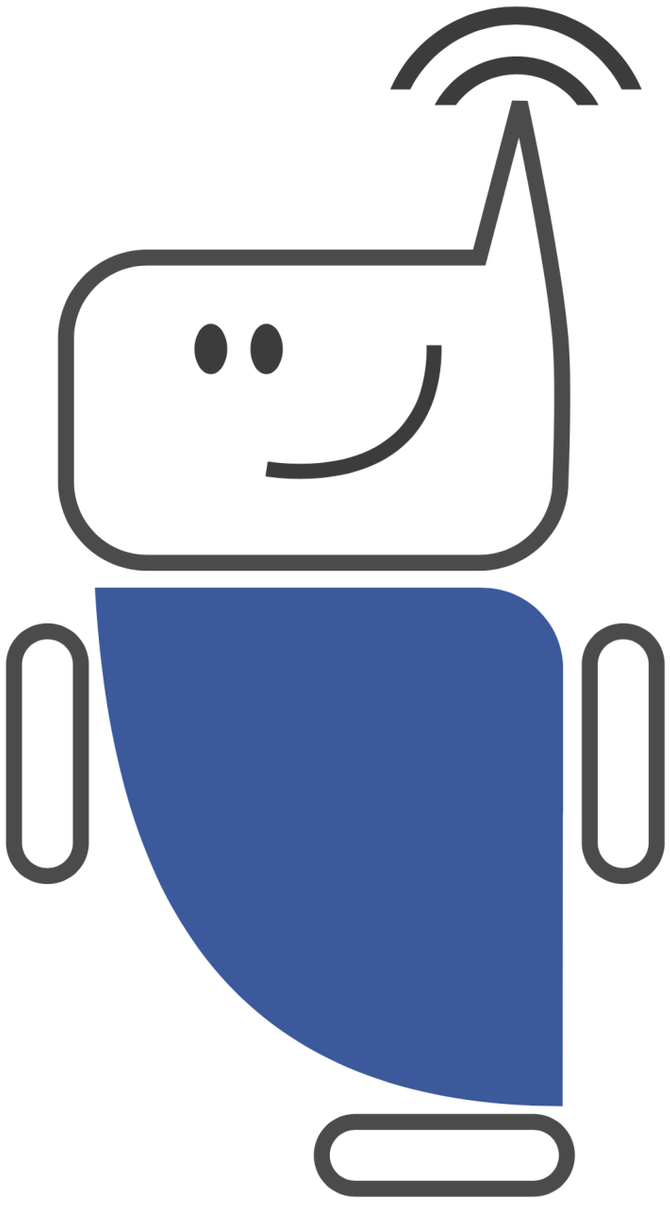
RETORNOS DE MÉTODOS E STRINGS FORMATADAS

Exemplo:

- Uma idade é muito mais versátil, no código, como **`number`**, ou seja, o próprio número.
- Do que como uma **`string`** formatada: **`"27 anos"`**.
- Comparar se uma pessoa atingiu maioridade:
  - Se o retorno for do tipo **`number`**, isso é fácil: **`27`**.
  - Caso seja a **`string`** formatada, demanda tratamento para extrair o número da cadeia de caracteres: **`"27 anos"`**.

CLASSES DE INTERFACE COM O USUÁRIO

- Deve haver **classes específicas para modelar a interação** com o usuário (classes de interface):
  - Instruções de entrada.
  - Instruções de saída.
- Deve-se prezar para que nenhuma instrução de entrada ou de saída esteja presente nas **classes que modelam a lógica da solução**.
  - Nenhum `console.log()`.
  - Nenhum `question()` e derivados.
- Fundamentos de Programação **não enfocará** as classes de interação com o usuário.



TESTES

TESTES AUTOMATIZADOS

- Frameworks Jest e Japa são exemplos.
- Maneira bastante **completa e inteligente** de testar objetos.
- Cria-se um conjunto de **testes** com o comportamento previsto dos objetos.
- É possível submeter os objetos a esse conjunto de testes **sem retrabalho**.
  - De testar, de forma manual, novamente.
- Os testes acabam servindo como um **protocolo** sobre como os objetos deveriam funcionar.
- Existem abordagens de desenvolvimento em que os testes são escritos **antes da funcionalidade em si**.
- Aprenderemos a utilizá-los na sequência da disciplina.

INSTANCIAÇÃO PELO ARQUITO INDEX.TS

- Maneira **simples** e, diga-se, **descartável** de testar o comportamento de objetos.
- Basta **instanciar e interagir** com os objetos em `index.ts`.
- O comportamento precisa ser **analisado por meio da interação**.
  - Exige tempo e atenção de quem programa.
- Os testes não precisam ser repetidos manualmente.
  - Mas também não são consistentemente automatizados como no Jest.
- Maneira **rápida** mas, até, **negligente** de testar o comportamento.
  - É possível, mas nem sempre apropriado.

INSTANCIAÇÃO PELO ARQUITO INDEX.TS

```
import teclado from "npm:readline-sync";
import Dosador from "./Dosador";

let peso: number = 0;

console.log("Digite o peso em kg:");
peso = teclado.questionFloat();

let dosador: Dosador = new Dosador(peso);

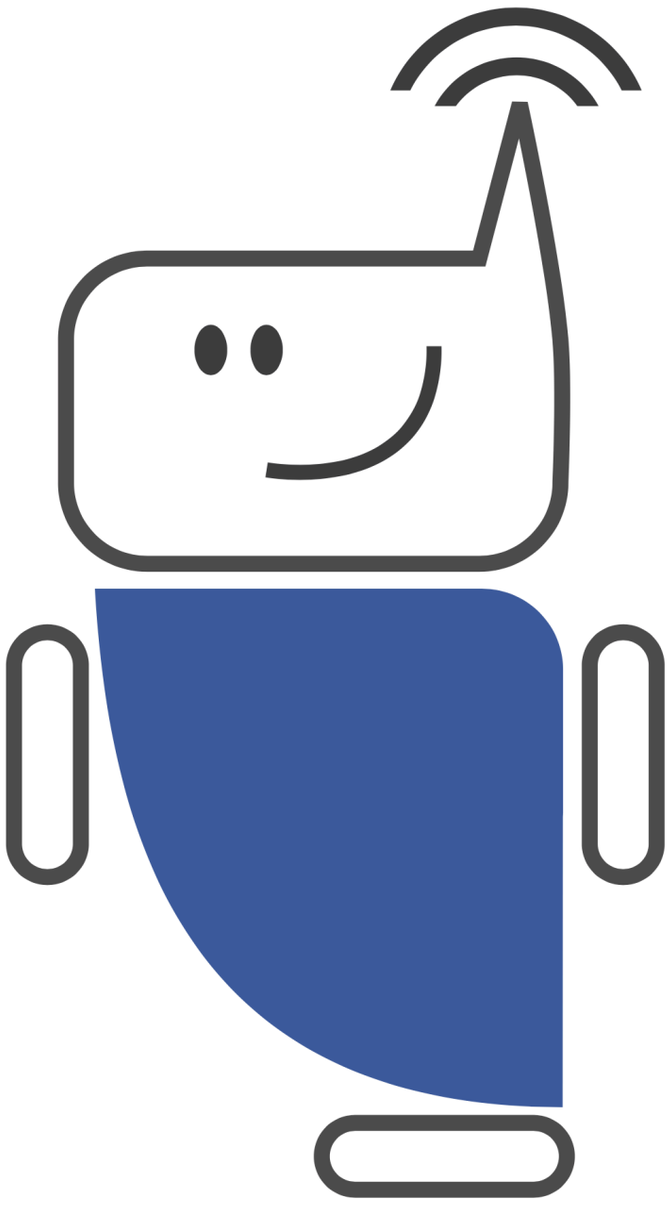
console.log("Dosagem:", dosador.getDosagem(), "ml");
```

INSTANCIAÇÃO PELO ARQUITO INDEX.TS

```
import Dosador from "./Dosador";

let peso: number = 72;
let umDosador: Dosador = new Dosador(peso);
console.log("Dosagem para uma pessoa de", peso, "kg:", umDosador.getDosagem(), "ml");

peso = 56;
let outroDosador: Dosador = new Dosador(peso);
console.log("Dosagem para uma pessoa de", peso, "kg:", outroDosador.getDosagem(), "ml");
```



OBJETOS EM

REPRESENTAÇÃO TEXTUAL

MÉTODO TOSTRING( )

- Considere o seguinte código, que instancia e imprime um objeto da classe **Dosador**.

```
let dosador: Dosador = new Dosador(72.6);
console.log(dosador.toString());
```

- O método de impressão invocou o método **toString()** daquele objeto.
- A saída em tela será:

```
[object Object]
```

- E, considerando a seguinte linha:

```
console.log(dosador);
```

- A saída em tela será:

```
Dosador { peso: 72.6 }
```

MÉTODO TOSTRING( )

— MAS A MINHA CLASSE NÃO TEM O MÉTODO TOSTRING( ). EU NÃO IMPLEMENTEI!

— TEM, SIM!

MÉTODO TOSTRING( )

- O método **`toString()`** é definido e implementado na classe **`Object`**.
  - Classe da qual descendem todas as outras.
  - Caso o **`toString()`** não seja sobrescrito (definido) pela classe filha, herda-se a implementação do método **`toString()`** da classe mãe.

- Por esse motivo, o **`toString()`** funciona sem estar implementado na classe **`Dosador`**.
  - Considera-se a implementação da classe **`Object`**.
  - Destaca-se que é possível **sobrescrevê-lo** para atender às necessidades do contexto.

MÉTODO TOSTRING( )

Conforme a documentação do TypeScript, o método **`toString()`**:

- Retorna uma **`string`** que **representa textualmente o objeto** naquele momento.
  - Características importantes do **estado do objeto**.
- E o resultado precisa ser uma representação **concisa, mas informativa, e fácil para que uma pessoa leia**.
- É recomendado que toda classe sobrescreva (**implemente novamente**) esse método.

MÉTODO TOSTRING( )

```
export default class Dosador {
  private peso: number;

  public constructor(peso: number) {
    this.peso = peso;
  }

  public getPeso(): number {
    return this.peso;
  }

  public setPeso(peso: number): void {
    this.peso = peso;
  }

  public getDosagem(): number {
    return (Math.ceil(this.peso / 10));
  }
}
```

SE UM OBJETO DOSADOR FOSSE REPRESENTADO COMO UMA STRING, O QUE ESSE TEXTO DEVERIA CONTER?

MÉTODO TOSTRING( )

```
export default class Dosador {
  private peso: number;

  public constructor(peso: number) {
    this.peso = peso;
  }

  public getPeso(): number {
    return this.peso;
  }

  public setPeso(peso: number): void {
    this.peso = peso;
  }

  public getDosagem(): number {
    return (Math.ceil(this.peso / 10));
  }

  public toString(): string {
    return "Dosagem para paciente com "
      + this.peso.toFixed(1) + " kg: "
      + this.getDosagem() + " ml";
  }
}
```

REPRESENTAÇÃO CONCISA, INFORMATIVA E FÁCIL PARA QUE UMA PESSOA LEIA.

MÉTODO TOSTRING( )

- Agora, diante da mesma interação com o objeto:

```
let dosador: Dosador = new Dosador(72.6);
console.log(dosador.toString());
```

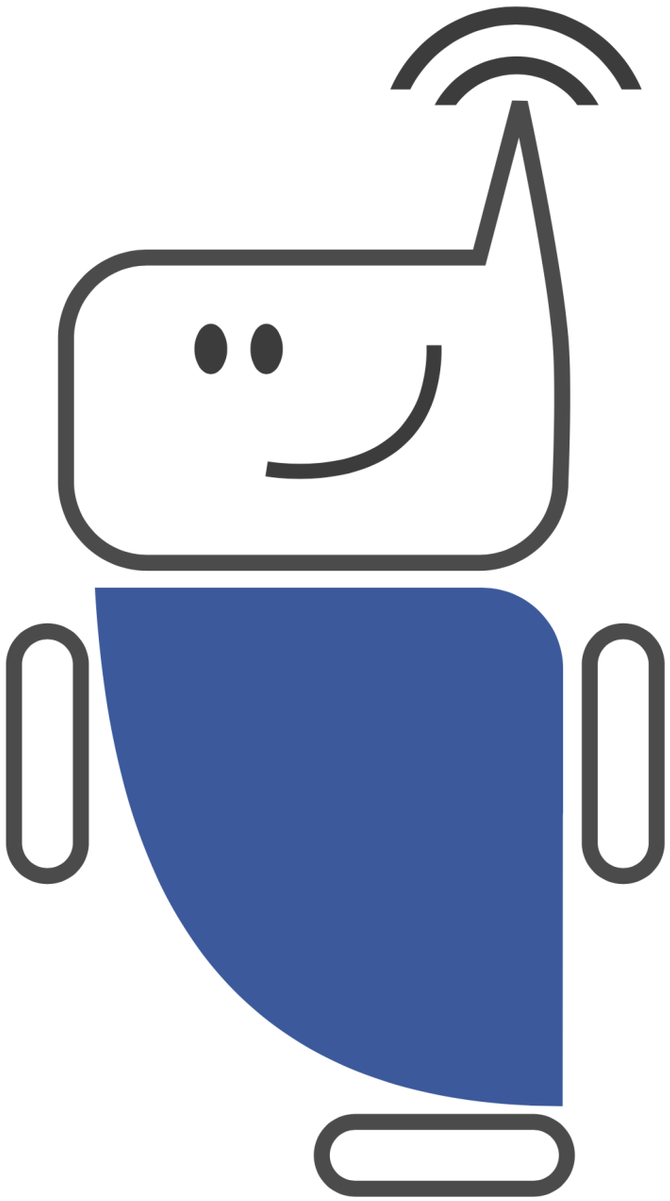
- É invocado o método **`toString()`** sobrescrito.

- E a saída em tela será:

```
Dosagem para paciente com 72,6 kg: 8 ml
```

- **Não se deve inventar de incluir, no texto, todos os atributos e/ou retornos de métodos, para qualquer objeto!**

```
let numero: number = 22;
console.log(numero.toString());
```



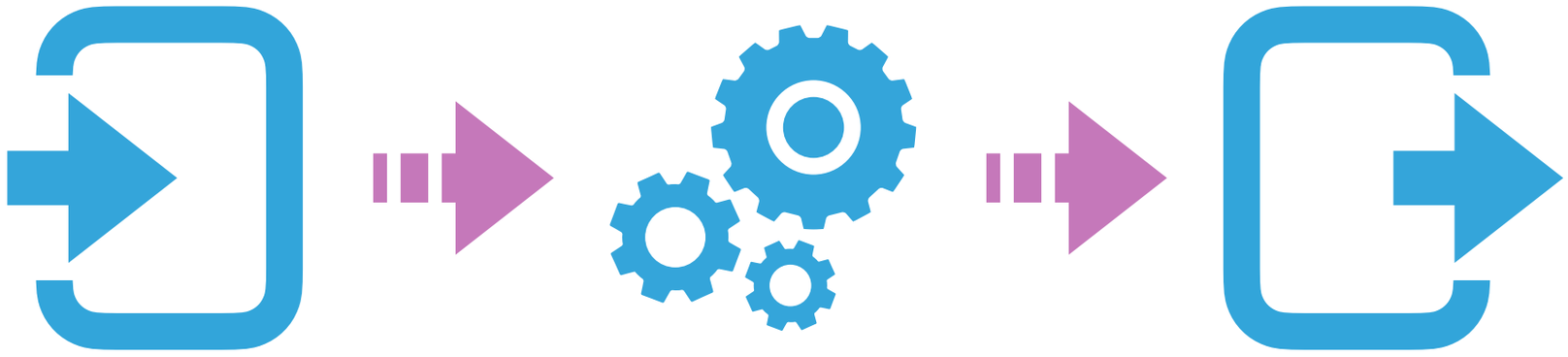
PRIMEIRAS MODELAGENS EM UM

ROTEIRO

1 ABSTRAIA E RESOLVA O PROBLEMA

- Qual é o **problema** que precisa ser solucionado?
- Qual é o **objetivo** que deve ser atingido?
- Quais são as **entradas**, o **processamento** e as **saídas**?
- Leia atentamente o enunciado!
- Não faça suposições.
- Rascunhe indispensavelmente!

1 ABSTRAIA E RESOLVA O PROBLEMA



ENTRADA

O que é fornecido? (*input*)

PROCESSAMENTO

Como transformo o que é fornecido no que deve ser retornado?

SAÍDA

O que é retornado? (*output*)

2 IDENTIFIQUE AS CLASSES E OS OBJETOS

- Quais são as **classes** e **objetos** envolvidos na resolução?
- Seja simples e não crie uma complexidade desnecessária.
  - Isso interfere drasticamente na solução.
- Leia atentamente o enunciado!

3 IDENTIFIQUE OS ATRIBUTOS

- Quais são os **atributos** (características) dos objetos envolvidos?

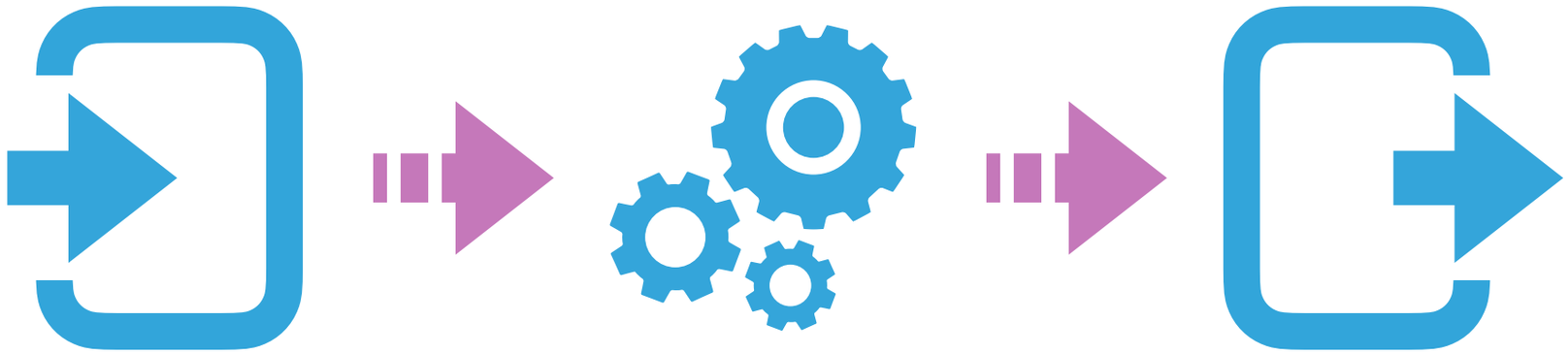
- Refine esses atributos de modo a definir respectivos:
  - Tipos.
  - Modificadores de acesso.

4 IDENTIFIQUE OS MÉTODOS

- Quais são os **métodos** (comportamentos) dos objetos envolvidos?

- Refine cada método, de modo a definir a respectiva assinatura (cabeçalho):
  - Parâmetros (e tipos).
  - Tipo de retorno.
  - Modificador de acesso.

4 IDENTIFIQUE OS MÉTODOS



ENTRADA

Parâmetros fornecidos ao método

PROCESSAMENTO

Operações executadas dentro do método

SAÍDA

Retorno fornecido pelo método

5 DEFINA O CONSTRUTOR

- Quais serão os **valores iniciais** dos atributos?
- Quais são os parâmetros que o construtor deve receber para **inicializar um objeto**?
- Quais são os **tipos** desses parâmetros?

6 IMPLEMENTE A SOLUÇÃO

- Implemente a solução (modelagem e algoritmos) em uma linguagem orientada a objetos.

- Codificação em TypeScript que amamos!

TERMINEI?

7 TESTE A SOLUÇÃO!

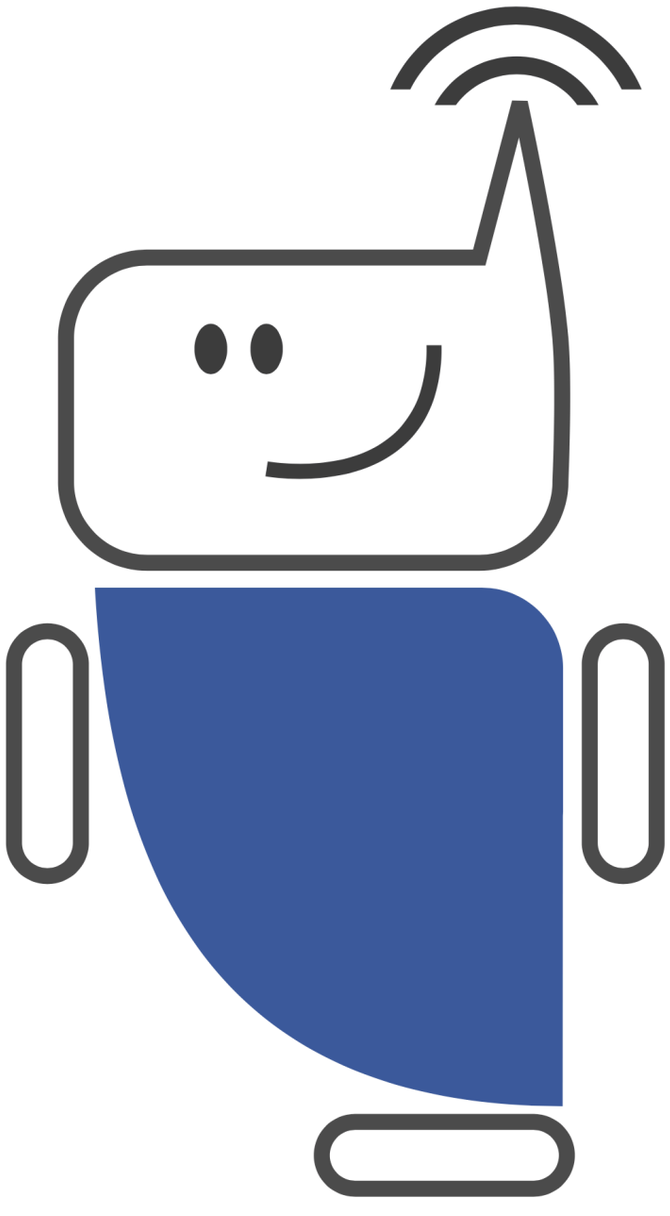
- Execute a solução.
- Se necessário, execute, **uma a uma**, as instruções do programa.
  - Siga **estritamente a ordem estabelecida** para verificar se os resultados esperados são produzidos.
- Detecte, por meio do **comportamento**, possíveis erros na solução.
  - Use um conjunto de testes variados.

8 APRIMORE A SOLUÇÃO

- É possível **melhorar** a modelagem ou os algoritmos?
- É possível **simplificar** a modelagem por meio da abstração?
- É possível **reduzir o número de passos** dados pelos algoritmos?
- É possível otimizar e fazer com uma quantidade **mínima de passos**?
- Diante de alterações, é necessário **testar a solução novamente**.

RESUMO

1. Abstraia e resolva o problema.
2. Identifique as classes e os objetos.
3. Identifique os atributos.
4. Identifique os métodos.
5. Defina o construtor.
6. Implemente a solução.
7. Teste a solução.
8. Aprimore a solução.



PRÓXIMOS PASSOS

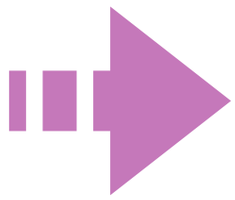
DISCIPLINA DE FUNDAMENTOS DE PROGRAMAÇÃO

- As disciplinas da Trilha de Programação do curso enfatizam a POO.
- Depois da AA1, a disciplina, passa a adotar o paradigma.
- De agora em diante, **somente serão admitidas soluções orientadas a objetos**.
  - Portanto, sem mais soluções sequenciais inteiras dentro de `index.ts`.
- A disciplina reservará à implementação das classes que **modelam a lógica das soluções**.
  - Não contemplará a implementação das interfaces.
  - Exceto quando especificado o contrário.
  - Maneira de proporcionar um aprendizado mais profundo na disciplina.

DISCIPLINA DE FUNDAMENTOS DE PROGRAMAÇÃO

EXEMPLO – DOSADOR

Modele e implemente um dosador automático para um medicamento destinado a idosos. A dosagem é prescrita conforme o peso do paciente (informado ao dosador), sendo de 1 ml para cada 10 kg. O dosador funciona com mililitros e não faz fracionamento. Portanto, uma pessoa de 55 kg recebe uma dosagem de 6 ml.



DOSADOR

```
- peso: number = 0.0
```

```
+ constructor(peso: number)
+ getPeso(): number
+ setPeso(peso: number): void
+ getDosagem(): number
```

DISCIPLINA DE FUNDAMENTOS DE PROGRAMAÇÃO

EXEMPLO – DOSADOR

Modele e implemente um dosador automático para um medicamento destinado a idosos. A dosagem é prescrita conforme o peso do paciente (informado ao dosador), sendo de 1 ml para cada 10 kg. O dosador funciona com mililitros e não faz fracionamento. Portanto, uma pessoa de 55 kg recebe uma dosagem de 6 ml.

```
export default class Dosador {
  private peso: number;

  public constructor(peso: number) {
    this.peso = peso;
  }

  public getPeso(): number {
    return this.peso;
  }

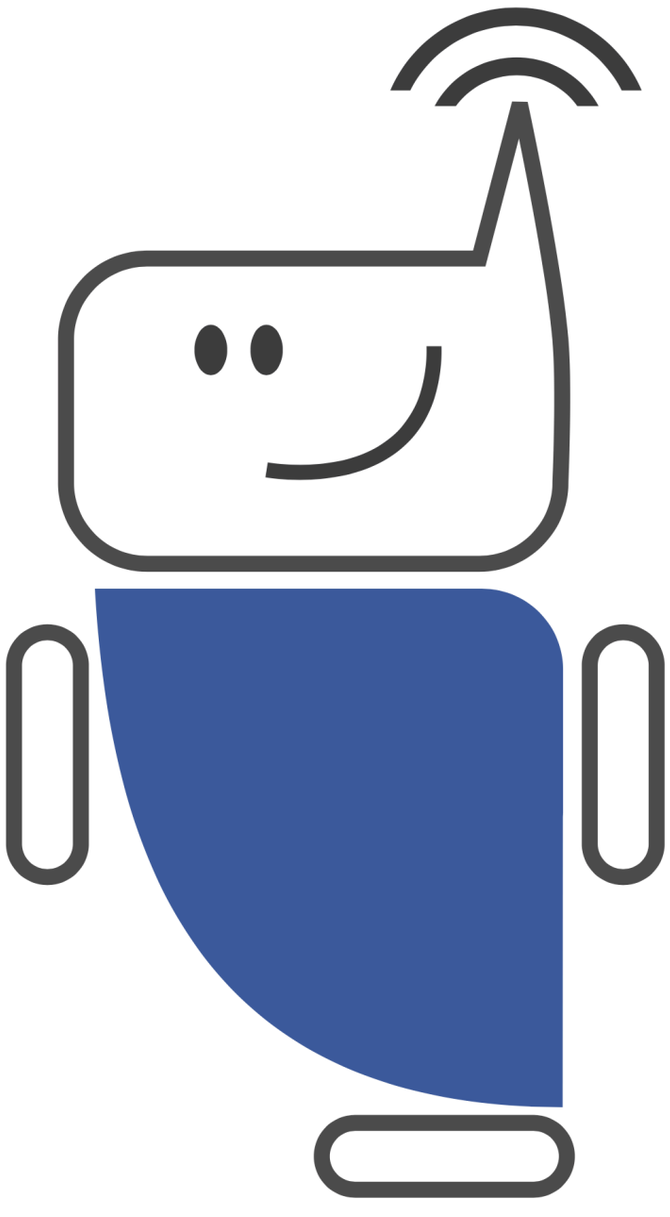
  public setPeso(peso: number): void {
    this.peso = peso;
  }

  public getDosagem(): number {
    return (Math.ceil(this.peso / 10));
  }
}
```

DISCIPLINA DE FUNDAMENTOS DE PROGRAMAÇÃO



Fonte: Reddit



REFERÊNCIAS

REFERÊNCIA

MASCHIO, Eleandro. **Programação Orientada a Objetos**. Guarapuava: Universidade Tecnológica Federal do Paraná, 2023. 70 slides, color. Material didático da disciplina de Fundamentos de Programação.

CITAÇÃO COM AUTOR INCLUÍDO NO TEXTO

Maschio (2023)

CITAÇÃO COM AUTOR NÃO INCLUÍDO NO TEXTO

(MASCHIO, 2023)